Graph Data Structure Study Guide

1. Connected Components

What are Connected Components?

A connected component in an undirected graph is a maximal set of vertices such that there is a path between every pair of vertices. In simpler terms, it's a group of nodes that are all connected to each other, either directly or through other nodes in the same group.

Key Concepts

Connected Graph: A graph where there is a path between every pair of vertices.

Disconnected Graph: A graph that has two or more connected components.

Strongly Connected Components (for directed graphs): A maximal set of vertices where there is a directed path from every vertex to every other vertex in the set.

Finding Connected Components using DFS

java

```java
import java.util.*;

public class ConnectedComponents {
    private int vertices;
    private List<List<Integer>> adjacencyList;
    private boolean[] visited;

    public ConnectedComponents(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();
        this.visited = new boolean[vertices];

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source); // For undirected graph
    }

    // DFS traversal for a connected component
    private void dfs(int vertex, List<Integer> component) {
        visited[vertex] = true;
        component.add(vertex);

        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                dfs(neighbor, component);
            }
        }
    }

    // Find all connected components
    public List<List<Integer>> findConnectedComponents() {
        List<List<Integer>> components = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, component);
                components.add(component);
            }
        }
```

```java
        return components;
    }

    public static void main(String[] args) {
        ConnectedComponents graph = new ConnectedComponents(6);

        // Add edges
        graph.addEdge(0, 1);
        graph.addEdge(1, 2);
        graph.addEdge(3, 4);

        List<List<Integer>> components = graph.findConnectedComponents();

        System.out.println("Connected Components:");
        for (int i = 0; i < components.size(); i++) {
            System.out.println("Component " + (i + 1) + ": " + components.get(i));
        }
    }
}
```

Finding Connected Components using BFS

java

```java
import java.util.*;

public class ConnectedComponentsBFS {
    private int vertices;
    private List<List<Integer>> adjacencyList;
    private boolean[] visited;

    public ConnectedComponentsBFS(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();
        this.visited = new boolean[vertices];

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source);
    }

    // BFS traversal for a connected component
    private void bfs(int startVertex, List<Integer> component) {
        Queue<Integer> queue = new LinkedList<>();
        visited[startVertex] = true;
        queue.offer(startVertex);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            component.add(vertex);

            for (int neighbor : adjacencyList.get(vertex)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
    }

    public List<List<Integer>> findConnectedComponents() {
        List<List<Integer>> components = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            if (!visited[i]) {
```

```
                List<Integer> component = new ArrayList<>();
                bfs(i, component);
                components.add(component);
            }
        }

        return components;
    }
}
```

Time and Space Complexity

Time Complexity: O(V + E) where V is the number of vertices and E is the number of edges.

Space Complexity: O(V) for the visited array and recursion stack (DFS) or queue (BFS).

2. Practice Questions on DFS/BFS

DFS Implementation and Variations

Basic DFS Traversal

java

```java
public class GraphDFS {
    private int vertices;
    private List<List<Integer>> adjacencyList;
    private boolean[] visited;

    public GraphDFS(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();
        this.visited = new boolean[vertices];

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
    }

    // Recursive DFS
    public void dfs(int vertex) {
        visited[vertex] = true;
        System.out.print(vertex + " ");

        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                dfs(neighbor);
            }
        }
    }

    // Iterative DFS using Stack
    public void dfsIterative(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Stack<Integer> stack = new Stack<>();

        stack.push(startVertex);

        while (!stack.isEmpty()) {
            int vertex = stack.pop();

            if (!visited[vertex]) {
                visited[vertex] = true;
                System.out.print(vertex + " ");

                // Add neighbors to stack in reverse order
```

```
                List<Integer> neighbors = adjacencyList.get(vertex);
                for (int i = neighbors.size() - 1; i >= 0; i--) {
                    if (!visited[neighbors.get(i)]) {
                        stack.push(neighbors.get(i));
                    }
                }
            }
        }
    }
}
```

Path Finding using DFS

```java
public boolean hasPath(int source, int destination) {
    boolean[] visited = new boolean[vertices];
    return dfsPath(source, destination, visited);
}

private boolean dfsPath(int current, int destination, boolean[] visited) {
    if (current == destination) {
        return true;
    }

    visited[current] = true;

    for (int neighbor : adjacencyList.get(current)) {
        if (!visited[neighbor]) {
            if (dfsPath(neighbor, destination, visited)) {
                return true;
            }
        }
    }

    return false;
}
```

BFS Implementation and Variations

Basic BFS Traversal

```java
public class GraphBFS {
    private int vertices;
    private List<List<Integer>> adjacencyList;

    public GraphBFS(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
    }

    public void bfs(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new LinkedList<>();

        visited[startVertex] = true;
        queue.offer(startVertex);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print(vertex + " ");

            for (int neighbor : adjacencyList.get(vertex)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
    }
}
```

Shortest Path using BFS (Unweighted Graph)

```java
public int shortestPath(int source, int destination) {
    if (source == destination) return 0;

    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();
    Queue<Integer> distances = new LinkedList<>();

    visited[source] = true;
    queue.offer(source);
    distances.offer(0);

    while (!queue.isEmpty()) {
        int vertex = queue.poll();
        int distance = distances.poll();

        for (int neighbor : adjacencyList.get(vertex)) {
            if (neighbor == destination) {
                return distance + 1;
            }

            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
                distances.offer(distance + 1);
            }
        }
    }

    return -1; // No path found
}
```

Level Order Traversal using BFS

```java
public void levelOrderTraversal(int startVertex) {
    boolean[] visited = new boolean[vertices];
    Queue<Integer> queue = new LinkedList<>();
    Queue<Integer> levels = new LinkedList<>();

    visited[startVertex] = true;
    queue.offer(startVertex);
    levels.offer(0);

    int currentLevel = 0;

    while (!queue.isEmpty()) {
        int vertex = queue.poll();
        int level = levels.poll();

        if (level > currentLevel) {
            System.out.println(); // New line for new level
            currentLevel = level;
        }

        System.out.print(vertex + " ");

        for (int neighbor : adjacencyList.get(vertex)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
                levels.offer(level + 1);
            }
        }
    }
}
```

Cycle Detection

Cycle Detection in Undirected Graph using DFS

```java
public boolean hasCycleUndirected() {
    boolean[] visited = new boolean[vertices];

    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            if (dfsHasCycle(i, -1, visited)) {
                return true;
            }
        }
    }

    return false;
}

private boolean dfsHasCycle(int vertex, int parent, boolean[] visited) {
    visited[vertex] = true;

    for (int neighbor : adjacencyList.get(vertex)) {
        if (!visited[neighbor]) {
            if (dfsHasCycle(neighbor, vertex, visited)) {
                return true;
            }
        } else if (neighbor != parent) {
            return true; // Back edge found
        }
    }

    return false;
}
```

Cycle Detection in Directed Graph using DFS

```java
public boolean hasCycleDirected() {
    boolean[] visited = new boolean[vertices];
    boolean[] recursionStack = new boolean[vertices];

    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            if (dfsHasCycleDirected(i, visited, recursionStack)) {
                return true;
            }
        }
    }

    return false;
}

private boolean dfsHasCycleDirected(int vertex, boolean[] visited, boolean[] recursionStack) {
    visited[vertex] = true;
    recursionStack[vertex] = true;

    for (int neighbor : adjacencyList.get(vertex)) {
        if (!visited[neighbor]) {
            if (dfsHasCycleDirected(neighbor, visited, recursionStack)) {
                return true;
            }
        } else if (recursionStack[neighbor]) {
            return true; // Back edge in directed graph
        }
    }

    recursionStack[vertex] = false;
    return false;
}
```

Quiz Questions

Question 1

In an undirected graph with 8 vertices, what is the maximum number of connected components possible?

A) 4

B) 6

C) 8

D) 7

Correct Answer: C) 8

Question 2

Which traversal algorithm is best suited for finding the shortest path in an unweighted graph?

A) DFS (Depth-First Search)
B) BFS (Breadth-First Search)
C) Both DFS and BFS
D) Neither DFS nor BFS

Correct Answer: B) BFS (Breadth-First Search)

Question 3

What is the time complexity of finding all connected components in a graph using DFS?

A) O(V)
B) O(E)
C) O(V + E)
D) O(V × E)

Correct Answer: C) O(V + E)

Question 4

In cycle detection for an undirected graph using DFS, a cycle is detected when:

A) We visit a vertex that is already visited
B) We visit a vertex that is already visited and is not the parent of the current vertex
C) We reach a leaf node
D) The recursion stack becomes empty

Correct Answer: B) We visit a vertex that is already visited and is not the parent of the current vertex

Question 5

Which data structure is used in the iterative implementation of BFS?

A) Stack
B) Queue
C) Priority Queue
D) Array

Correct Answer: B) Queue

Practice Coding Questions

Question 1: Number of Islands

Given a 2D binary matrix filled with 0's and 1's, find the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the matrix are surrounded by water.

Example:

```
Input: matrix = [
  ['1','1','1','1','0'],
  ['1','1','0','1','0'],
  ['1','1','0','0','0'],
  ['0','0','0','0','0']
]
Output: 1
```

Question 2: Clone Graph

Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node in the graph contains a value (int) and a list of its neighbors.

Note: The graph is represented using an adjacency list. For simplicity, each node's value is the same as the node's index (1-indexed).

Question 3: Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai. Return true if you can finish all courses. Otherwise, return false.

Example:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take. To take course 1 you should have
finished course 0. So it is possible.
```

Question 4: Word Ladder

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that every adjacent pair of words differs by a single letter, and every si is in wordList. Given two words, beginWord and endWord, and a dictionary wordList, return the length of the shortest transformation sequence from beginWord to endWord, or 0 if no such sequence exists.

Question 5: Surrounded Regions

Given an m x n matrix board containing 'X' and 'O', capture all regions that are 4-directionally surrounded by 'X'. A region is captured by flipping all 'O's into 'X's in that surrounded region.